

## TLS 1.3 · QWAC / QSEAL · JWS / JWE · APub / APriv

Application aux endpoints BankID et au stockage KYC

|                  |                                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Source</b>    | <a href="https://developer.bankid.cz/docs/security_sep">developer.bankid.cz/docs/security_sep</a> (v1.3 – 10.04.2025) |
| <b>Périmètre</b> | Service Provider (SeP) – consommateur BankID                                                                          |
| <b>Stack</b>     | Java 17 · Spring Boot 4 · Nimbus JOSE+JWT · Bouncy Castle                                                             |
| <b>Date</b>      | Mai 2026                                                                                                              |

# 1. Vue d'ensemble — Les 4 mécanismes de sécurité

La documentation **security\_sep** de BankID définit quatre mécanismes de sécurité distincts que le SeP doit maîtriser. Ce document décrit chacun, sa source textuelle dans la doc officielle, et son application concrète sur chaque endpoint et sur le stockage KYC.

| Mécanisme    | Obligation (RFC2119)     | Rôle principal                                                      |
|--------------|--------------------------|---------------------------------------------------------------------|
| TLS 1.3      | MUST (depuis 01/07/2025) | Chiffrement du transport — toutes les communications                |
| QWAC         | RECOMMEND (SHOULD)       | Certificat serveur qualifié sur la Gateway                          |
| QSEAL        | RECOMMEND (SHOULD)       | Certificat de signature des JWT envoyés par BankID                  |
| JWS / JWE    | MAY (si activé → MUST)   | Signature et chiffrement des réponses /userinfo, /profile, id_token |
| APub / APriv | MUST                     | Chiffrement asymétrique des payloads d'audit KYC (15 ans)           |

## ■ Point clé sur TLS 1.3

La doc indique explicitement : TLS 1.2 était toléré jusqu'au 1er juillet 2025.

Depuis cette date, TLS 1.2 est supprimé. TLS 1.3 est maintenant la version minimale obligatoire.

Tout appel WebClient vers BankID DOIT forcer TLS 1.3. Les configurations TLS 1.2 existantes sont non conformes.

## 2. TLS 1.3

### 2.1 Ce que dit la doc (source exacte)

#### ■ Extrait security\_sep §TLS

« All communication MUST use a secure version of TLS. It is possible to use TLS 1.2 until 1.7.2025, from 1.7.2025 support for TLS 1.2 will be removed. »

« The server MUST NOT allow protocol renegotiation to an older/weaker version of TLS than 1.3. »

« TLS compression MUST be disabled. » (anti-CRIME)

« HTTP compression MUST be DISABLED. » (anti-BREACH)

### 2.2 Application par endpoint

| Endpoint BankID                  | Direction           | TLS 1.3 requis | Note                             |
|----------------------------------|---------------------|----------------|----------------------------------|
| /well-known/openid-configuration | SeP → BankID        | MUST           | Auto-découverte OIDC             |
| /well-known/jwks                 | SeP → BankID        | MUST           | Fetch des clés publiques JWK     |
| /auth                            | Navigateur → BankID | MUST           | Redirect utilisateur             |
| /token                           | SeP → BankID        | MUST           | Échange code + refresh           |
| /userinfo                        | SeP → BankID        | MUST           | Récupération claims KYC          |
| /profile                         | SeP → BankID        | MUST           | Claims KYC complets (JWT)        |
| /revoke                          | SeP → BankID        | MUST           | Révocation tokens                |
| /callback                        | BankID → Gateway    | MUST           | Retour code d'autorisation       |
| API interne Back↔Gateway         | Interne             | SHOULD         | Réseau privé, TLS 1.3 recommandé |

### 2.3 Configuration WebClient Spring Boot 4

```
// BankIdWebClientConfig.java – TLS 1.3 forcé
@Bean
public WebClient bankIdWebClient() throws Exception {
    SslContext sslContext = SslContextBuilder.forClient()
        .protocols("TLSv1.3") // TLS 1.3 uniquement
        .ciphers(null) // Ciphers par défaut TLS 1.3
        .trustManager(buildTrustManager()) // TrustStore avec CA BankID
        .build();

    HttpClient httpClient = HttpClient.create()
        .secure(spec -> spec.sslContext(sslContext)
            .handshakeTimeout(Duration.ofSeconds(10)))
        .compress(false) // MUST : désactiver compression
        .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 5000);

    return WebClient.builder()
        .baseUrl(bankIdBaseUrl)
        .clientConnector(new ReactorClientHttpConnector(httpClient))
        .defaultHeader("Accept-Encoding", "") // MUST : désactiver HTTP compression
        .build();
}

private TrustManagerFactory buildTrustManager() throws Exception {
    KeyStore ts = KeyStore.getInstance("PKCS12");
    ts.load(truststorePath.getInputStream(), tsPassword.toCharArray());
    TrustManagerFactory tmf = TrustManagerFactory.getInstance(
        TrustManagerFactory.getDefaultAlgorithm());
    tmf.init(ts);
    return tmf;
}
```

## 2.4 Configuration Gateway (Spring Cloud Gateway)

```
# application.yml – Gateway
server:
  ssl:
    enabled: true
    protocol: TLSv1.3
    enabled-protocols: TLSv1.3
    key-store: classpath:certs/gateway-keystore.p12
    key-store-password: ${GATEWAY_KS_PASSWORD}
    key-store-type: PKCS12

# Headers de sécurité obligatoires (si mTLS activé entre gateway et BankID)
spring:
  cloud:
    gateway:
      default-filters:
        - AddResponseHeader=Strict-Transport-Security,max-age=31536000; includeSubDomains
        - AddResponseHeader=Content-Security-Policy,default-src 'self'
        - AddResponseHeader=X-Content-Type-Options,nosniff
        - AddResponseHeader=X-XSS-Protection,1; mode=block
        - RemoveResponseHeader=Server
        - RemoveResponseHeader=X-Powered-By
```

## 3. QWAC et QSEAL

### 3.1 Ce que dit la doc (source exacte)

#### ■ Extrait security\_sep §Certificates

« We RECOMMEND the use of QWAC certificates for server authentication. »

« We RECOMMEND the use of QSEAL certificates for JSON Web Signatures in payloads. »

« Signing keys in Bank ID are always accompanied by QSEAL certificate i.e. signature of JWT is always done by QSEAL. QSEAL certificate might be issued by any certification authority in EU Trusted list! »

« Server certificates MUST be rotated regularly, at least once a year. »

« An SSL Wildcard certificate MUST NOT be used for server authentication. »

### 3.2 Distinction QWAC vs QSEAL

|                | QWAC                                          | QSEAL                                         |
|----------------|-----------------------------------------------|-----------------------------------------------|
| Signification  | Qualified Website Authentication Certificate  | Qualified Electronic Seal Certificate         |
| Rôle           | Authentification TLS du serveur (HTTPS)       | Signature numérique des payloads JWT          |
| Qui l'utilise  | Ton serveur (Gateway)                         | BankID (signe les JWT qu'il t'envoie)         |
| Obligation SeP | RECOMMEND (SHOULD)                            | SHOULD pour tes propres signatures            |
| Délivré par    | CA de l'EU Trust List                         | CA de l'EU Trust List                         |
| Algorithme     | ECDSA ≥ 256 bits (préféré) ou RSA ≥ 2048 bits | ECDSA ≥ 256 bits (préféré) ou RSA ≥ 3072 bits |
| Rotation       | Au moins une fois par an (MUST)               | Au moins une fois par an (MUST)               |

### 3.3 QWAC — Application sur la Gateway

Le QWAC remplace le certificat TLS standard sur ta Gateway. C'est le certificat que les clients et BankID voient lorsqu'ils établissent une connexion HTTPS avec ton service.

```
# application.yml – Gateway avec certificat QWAC
server:
  ssl:
    enabled: true
    protocol: TLSv1.3
    # Le keystore contient la clé privée + certificat QWAC
    key-store: ${QWAC_KEYSTORE_PATH}
    key-store-password: ${QWAC_KS_PASSWORD}
    key-store-type: PKCS12
    key-alias: qwac-cert
    # TrustStore contenant les CA BankID (root + intermédiaires fournis dans la doc)
    trust-store: ${BANKID_TRUSTSTORE_PATH}
    trust-store-password: ${BANKID_TS_PASSWORD}
    # Wildcard MUST NOT être utilisé – certificat FQDN obligatoire
    # ex: kyc.votre-domaine.com (pas *.votre-domaine.com)
```

### 3.4 QSEAL — Utilisation côté BankID (ce que tu reçois)

BankID signe tous ses JWT avec son certificat QSEAL. Côté SeP, tu dois **vérifier** cette signature en utilisant les clés publiques exposées dans le JWK endpoint. La doc précise que le kid (Key ID) est toujours inclus dans le header JWT pour identifier la clé exacte à utiliser.

```
// BankIdJwkProvider.java – Récupération et cache des clés QSEAL BankID
@Component
public class BankIdJwkProvider {

    // Nimbus RemoteJWKSet : cache automatique + re-fetch si kid inconnu
    private final RemoteJWKSet<SecurityContext> jwkSet;

    public BankIdJwkProvider(@Value("${bankid.base-url}") String baseUrl)
        throws MalformedURLException {
        URL jwksUrl = new URL(baseUrl + "/.well-known/jwks");
        // DefaultResourceRetriever : timeout 5s, cache 10min
        this.jwkSet = new RemoteJWKSet<>(jwksUrl,
            new DefaultResourceRetriever(5000, 10000, 50 * 1024));
    }

    // Récupère la clé publique correspondant au kid du JWT reçu
    public RSAKey getSigningKey(String kid) throws KeySourceException {
        JWMatcher matcher = new JWMatcher.Builder()
            .keyID(kid).keyUse(KeyUse.SIGNATURE).build();
        List<JWK> keys = jwkSet.get(new JWKSelector(matcher), null);
        if (keys.isEmpty()) throw new JwtValidationException("Unknown kid: " + kid);
        return (RSAKey) keys.get(0);
    }
}
```

### 3.5 Rotation des clés — stratégie obligatoire

La doc indique que BankID peut déployer de nouvelles clés sans préavis. La stratégie de rotation doit être intégrée dans le code :

```
// Stratégie : re-fetch automatique sur kid inconnu
public JWTClaimsSet verifySignature(SignedJWT jwt) throws Exception {
    String kid = jwt.getHeader().getKeyID();
    try {
        RSAKey key = jwkProvider.getSigningKey(kid);
        RSASSAVerifier verifier = new RSASSAVerifier(key.toRSAPublicKey());
        if (!jwt.verify(verifier))
            throw new JwtValidationException("Signature invalide");
        return jwt.getJWTClaimsSet();
    } catch (KeySourceException e) {
        // kid inconnu : forcer un re-fetch du JWK endpoint
        jwkProvider.invalidateCache();
        RSAKey key = jwkProvider.getSigningKey(kid); // second essai
        RSASSAVerifier verifier = new RSASSAVerifier(key.toRSAPublicKey());
        if (!jwt.verify(verifier))
            throw new JwtValidationException("Signature invalide après rotation");
        return jwt.getJWTClaimsSet();
    }
}
```

### 3.6 TrustStore — Certificats CA BankID fournis dans la doc

La doc `security_sep` fournit directement les certificats root CA et intermédiaires de BankID (section «Bank iD SIGN root and intermediate certificates»). Ces certificats DOIVENT être importés dans ton TrustStore pour valider les signatures QSEAL reçues.

| Certificat                           | Validité    | Usage                          |
|--------------------------------------|-------------|--------------------------------|
| Bank iD Document Root CA             | 2021 → 2041 | CA racine — ancre de confiance |
| Bank iD Intermediate CA (2021-10-07) | 2021 → 2024 | CA intermédiaire (expiré)      |
| Bank iD Intermediate CA (2024-10-02) | 2024 → 2028 | CA intermédiaire actif         |
| Bank iD Intermediate CA (2024-12-01) | 2024 → 2029 | CA intermédiaire actif         |

```
# Importer les CA BankID dans le TrustStore (à faire une fois, rotation annuelle)
keytool -importcert -alias bankid-root-ca \
    -file bankid-root-ca.pem \
    -keystore bankid-truststore.p12 \
    -storetype PKCS12 \
    -storepass ${TRUSTSTORE_PASSWORD} -noprompt

keytool -importcert -alias bankid-intermediate-2024-10 \
    -file bankid-intermediate-2024-10.pem \
    -keystore bankid-truststore.p12 ...
```

## 4. JWS et JWE — Signature et Chiffrement des Réponses

### 4.1 Ce que dit la doc (source exacte)

#### ■ Extrait security\_sep §Authentication (OpenID Connect)

« The Provider MAY use JWS and JWE where the OpenID connect protocol allows it: »

- ID Token in token endpoint response (signed and encrypted)
- UserInfo endpoint response (signed and encrypted)
- Profile endpoint response (signed and encrypted)

« When Provider uses JWE and JWS MUST support encryption and signature of request objects »

« Encryption MUST be done after the signature » (ordre : sign d'abord, encrypt ensuite)

« JWT from KYC and other future APIs MUST be first signed by the OpenID Producer key »

« and then encrypted by the OpenID Consumer key. »

### 4.2 Comprendre la structure JWS dans JWE

Lorsque BankID active JWS+JWE sur un endpoint (configuré dans ton application sur le portail), la réponse est un JWT doublement protégé :

| Couche            | Opération               | Clé utilisée               | Objectif                   |
|-------------------|-------------------------|----------------------------|----------------------------|
| 1 — JWS (interne) | BankID signe le payload | Clé privée QSEAL BankID    | Non-répudiation, intégrité |
| 2 — JWE (externe) | BankID chiffre le JWS   | Ta clé publique (jwks_uri) | Confidentialité            |
| Réception SeP     | Tu déchiffres le JWE    | Ta clé privée              | Obtenir le JWS             |
| Vérification SeP  | Tu vérifies le JWS      | Clé publique BankID (JWK)  | Authentifier l'origine     |

### 4.3 Application par endpoint

| Endpoint               | JWS possible        | JWE possible            | Content-Type si activé | Action SeP                     |
|------------------------|---------------------|-------------------------|------------------------|--------------------------------|
| /token → id_token      | Oui (STRONG CIPHER) | Oui (RSA-OAE P+A256GCM) | application/jwt        | Déchiffrer JWE → vérifier JWS  |
| /userinfo              | Oui                 | Oui                     | application/jwt        | Idem                           |
| /profile               | Oui                 | Oui                     | application/jwt        | Idem (données KYC complètes)   |
| /token → access_token  | Non (opaque)        | Non                     | —                      | Utiliser tel quel comme Bearer |
| /token → refresh_token | Non (opaque)        | Non                     | —                      | Stocker chiffré (AES-GCM)      |

■ JWS/JWE est optionnel MAIS...



C'est BankID qui décide d'activer JWS/JWE selon la configuration de ton application.

Si ton application est configurée avec `jwtks_uri`, les réponses `/userinfo` et `/profile`

PEUVENT arriver sous forme de JWT (et non JSON brut). Ton code DOIT gérer les deux cas.

Test : vérifier le Content-Type de la réponse — `'application/jwt'` = JWT, `'application/json'` = JSON brut.

## 4.4 Service de déchiffrement JWE + vérification JWS

```

// JwtProcessingService.java
@Service
public class JwtProcessingService {

    private final RSAPrivateKey sepPrivateKey; // Ta clé privée (jwks_uri)
    private final BankIdJwkProvider jwkProvider; // Clés publiques BankID

    // Point d'entrée : détecte JSON brut ou JWT
    public Map<String, Object> processUserInfoResponse(String responseBody) {
        if (responseBody.trim().startsWith("{")) {
            // JSON brut – JWS/JWE non activé
            return objectMapper.readValue(responseBody, MAP_TYPE);
        }
        // JWT (JWE wrapping JWS)
        return processSignedEncryptedJwt(responseBody);
    }

    public Map<String, Object> processSignedEncryptedJwt(String jwtString) {
        // Étape 1 : Déchiffrement JWE avec ta clé privée
        EncryptedJWT encryptedJWT = EncryptedJWT.parse(jwtString);
        RSADecrypter decrypter = new RSADecrypter(sepPrivateKey);
        encryptedJWT.decrypt(decrypter);

        // Étape 2 : Extraction du JWS interne
        SignedJWT signedJWT = encryptedJWT.getPayload().toSignedJWT();
        if (signedJWT == null)
            throw new JwtValidationException("Payload n'est pas un JWS");

        // Étape 3 : Vérification signature JWS avec clé QSEAL BankID
        String kid = signedJWT.getHeader().getKeyID();
        RSAKey bankIdKey = jwkProvider.getSigningKey(kid);
        RSASSAVerifier verifier = new RSASSAVerifier(bankIdKey.toRSAPublicKey());
        if (!signedJWT.verify(verifier))
            throw new JwtValidationException("Signature JWS invalide");

        // Étape 4 : Validation des claims standards
        JWTClaimsSet claims = signedJWT.getJWTClaimsSet();
        validateStandardClaims(claims);

        return claims.getClaims();
    }

    private void validateStandardClaims(JWTClaimsSet claims) {
        // iss : doit correspondre à l'issuer BankID connu
        if (!bankIdIssuer.equals(claims.getIssuer()))
            throw new JwtValidationException("Issuer invalide");
        // exp : token non expiré
        if (claims.getExpirationTime().before(new Date()))
            throw new JwtValidationException("Token expiré");
        // aud : doit contenir ton client_id
        if (!claims.getAudience().contains(clientId))
            throw new JwtValidationException("Audience invalide");
    }
}

```

## 4.5 Exposition de ton propre JWKS (jwks\_uri)

Si JWE est activé, BankID a besoin de ta clé publique pour chiffrer les réponses. Tu dois exposer un endpoint JWKS public et HTTPS sur ta Gateway :

```
// JwksController.java – Exposé via Gateway sur GET /.well-known/jwks.json
@RestController
public class JwksController {

    @Value("${sep.encryption.public-key-path}")
    private Resource publicKeyPath;

    @GetMapping("/.well-known/jwks.json")
    public ResponseEntity<String> jwks() throws Exception {
        RSAKey rsaKey = RSAKey.parseFromPEMEncodedObjects(
            new String(publicKeyPath.getInputStream().readAllBytes()));

        // Exposer uniquement la clé publique (jamais la clé privée !)
        RSAKey publicOnly = rsaKey.toPublicJWK();
        JWKSet jwkSet = new JWKSet(publicOnly);

        return ResponseEntity.ok()
            .contentType(MediaType.APPLICATION_JSON)
            .cacheControl(CacheControl.maxAge(10, TimeUnit.MINUTES))
            .body(jwkSet.toString());
    }
}
```

## 4.6 Dépendances Maven pour JWS/JWE

```
<dependencies>
  <!-- Nimbus JOSE+JWT : JWS, JWE, JWK -->
  <dependency>
    <groupId>com.nimbusds</groupId>
    <artifactId>nimbus-jose-jwt</artifactId>
    <version>9.40</version>
  </dependency>
  <!-- Bouncy Castle : algorithmes EC (ES512) -->
  <dependency>
    <groupId>org.bouncycastle</groupId>
    <artifactId>bcpkix-jdk18on</artifactId>
    <version>1.78</version>
  </dependency>
</dependencies>
```

## 5. APub / APriv — Chiffrement Asymétrique d'Audit

### 5.1 Ce que dit la doc (source exacte)

#### ✗ Exigences MUST — Extrait security\_sep §Audit

- « A special key pair for auditing MUST exist. APub = public part, APriv = private part. »
- « APub MUST be used by the RP systems to encrypt payloads of Bank data messages. »
- « The payloads of the messages MUST NOT be stored in plaintext or be encrypted using a symmetric cipher. » → AES-256-GCM seul n'est PAS conforme pour les données d'audit.
- « APriv MUST be in cold storage and MUST be stored in a tamper-resistant device (HSM). »
- « Access to APriv MUST be limited and controlled by a strict security process. »
- « The Bank iD systems MUST NOT have access to APriv. »
- « Decryption by APriv MUST be a manual process and the act itself MUST be audited. »
- « Data MUST be auditable for a duration of 15 years. »

### 5.2 Pourquoi ce mécanisme remet en question le stockage AES-256

Dans la doc SeP précédente, le stockage des données KYC était prévu en AES-256-GCM (chiffrement symétrique). La doc security\_sep impose pour les **données d'audit** un chiffrement **asymétrique avec APub**. Il faut donc distinguer deux types de stockage :

| Type de données                                            | Chiffrement autorisé               | Justification                               |
|------------------------------------------------------------|------------------------------------|---------------------------------------------|
| Données KYC opérationnelles (accès fréquent, durée 30j)    | AES-256-GCM (symétrique) — OK      | Usage opérationnel, accès applicatif normal |
| Payloads d'audit (données brutes reçues de BankID, 15 ans) | RSA/EC + APub (asymétrique) — MUST | Exigence explicite de la doc security_sep   |
| Métadonnées d'audit (dates, subHash, txn)                  | Non chiffré ou AES — OK            | Pas de PII, pas de payload bancaire         |

### 5.3 Architecture APub / APriv

#### ■ Principe de fonctionnement

APub (clé publique) : sur ton serveur Back API. Utilisée pour chiffrer les payloads.

APriv (clé privée) : en COLD STORAGE dans un HSM. Jamais sur le serveur applicatif.

Chiffrement : automatique à chaque réception de données BankID (Back API).

Déchiffrement : processus manuel, uniquement en cas de contrôle légal, audité.

Durée : les payloads chiffrés avec APub sont conservés 15 ans minimum.

## 5.4 Génération de la paire APub / APriv

```
# Générer la paire de clés d'audit EC P-521 (STRONG CIPHER)
openssl ecparam -name secp521r1 -genkey -noout -out audit-apriv.pem
openssl ec -in audit-apriv.pem -pubout -out audit-apub.pem

# APriv → dans le HSM (ex: AWS CloudHSM, HashiCorp Vault HSM)
# APub → sur le serveur Back API (classpath:certs/audit-apub.pem)

# Ou en RSA 3072 bits :
openssl genrsa -out audit-apriv-rsa.pem 3072
openssl rsa -in audit-apriv-rsa.pem -pubout -out audit-apub-rsa.pem
```

## 5.5 Service de chiffrement d'audit (APub)

```
// AuditEncryptionService.java
@Service
public class AuditEncryptionService {

    // APub chargée au démarrage – fichier PEM sur le serveur
    @Value("classpath:certs/audit-apub.pem")
    private Resource auditPublicKeyResource;

    private ECPublicKey auditPublicKey;

    @PostConstruct
    public void init() throws Exception {
        String pem = new String(auditPublicKeyResource.getInputStream().readAllBytes());
        // Parsing via Nimbus ou Bouncy Castle
        ECKey ecKey = (ECKey) JWK.parseFromPEMEncodedObjects(pem);
        this.auditPublicKey = ecKey.toECPublicKey();
    }

    // Chiffre le payload brut reçu de BankID avec APub (ECDH-ES + A256GCM)
    public String encryptForAudit(String rawPayload) throws Exception {
        ECKey ecKey = new ECKey.Builder(Curve.P_521, auditPublicKey).build();

        JWEHeader header = new JWEHeader.Builder(
            JWEAlgorithm.ECDH_ES, EncryptionMethod.A256GCM)
            .keyID("audit-apub")
            .build();

        JWEObject jwe = new JWEObject(header,
            new Payload(rawPayload.getBytes(StandardCharsets.UTF_8)));
        jwe.encrypt(new ECDHEncrypter(ecKey));

        return jwe.serialize(); // Stocker cette chaîne – déchiffrement manuel uniquement
    }
}
```

## 5.6 Intégration dans le flux KYC — où appeler AuditEncryptionService

Étape du flux

Action audit

Timing

|                                         |                                                                    |                                 |
|-----------------------------------------|--------------------------------------------------------------------|---------------------------------|
| Réception réponse /userinfo ou /profile | encryptForAudit(rawResponseBody) → stocker dans audit/             | Immédiat, avant tout traitement |
| Échange /token → id_token reçu          | encryptForAudit(rawIdToken) → stocker                              | Immédiat après réception        |
| Purge données opérationnelles (30j)     | NE PAS purger les fichiers audit — conserver 15 ans                | Le batch ne touche pas /audit   |
| Contrôle légal (rare)                   | Processus manuel : sortir APriv du HSM, déchiffrer, auditer l'acte | Hors scope applicatif           |

## 5.7 Structure de stockage avec séparation audit / opérationnel

```

/data/kyc-files/
■■■■ operational/
■   ■■■■ {subHash}.kyc.enc      ← AES-256-GCM (accès applicatif, purge 30j)
■   ■■■■ {subHash}.meta.json   ← Métadonnées non chiffrées
■■■■ audit/
    ■■■■ {txnId}-userinfo.jwe    ← Chiffré APriv (ECDH-ES+A256GCM), 15 ans
    ■■■■ {txnId}-idtoken.jwe    ← Chiffré APriv, 15 ans
    ■■■■ purge-audit.log        ← Journal des purges (pas de PII)

```

```

// KycOrchestrator.java – intégration complète
public KycResult processCallback(String code, HttpSession session) {
    // 1. Échange code → tokens
    TokenResponse tokens = tokenService.exchangeCode(code);

    // 2. Audit id_token brut (MUST : avant déchiffrement)
    auditService.encryptAndStore(
        session.getId() + "-idtoken",
        tokens.getRawIdToken()
    );

    // 3. Appel /userinfo ou /profile
    String rawKycResponse = bankIdClient.fetchUserInfo(tokens.getAccessToken());

    // 4. Audit payload brut KYC (MUST)
    auditService.encryptAndStore(
        session.getId() + "-userinfo",
        rawKycResponse
    );

    // 5. Déchiffrement JWE + vérification JWS (si activé)
    Map<String, Object> claims = jwtProcessor.processUserInfoResponse(rawKycResponse);

    // 6. Stockage opérationnel chiffré AES-256-GCM (accès quotidien)
    kycStorageService.storeOperational(claims);

    return new KycResult(claims.get("sub").toString());
}

```

## 6. Headers de Sécurité TLS Obligatoires

### 6.1 Ce que dit la doc

#### ■ Extrait security\_sep §Headers

« If the communication between the OpenID Provider and the Consumer uses mTLS, the following headers MUST be set: »

- Strict-Transport-Security: max-age=; IncludeSubDomains
- Content-Security-Policy
- Access-Control-Allow-Origin
- X-XSS-Protection: 1; mode=block
- X-Content-Type-Options: nosniff

« Server versioning information SHOULD BE removed/masked: Server, X-Powered-By »

### 6.2 Configuration Spring Cloud Gateway

```
// SecurityHeadersFilter.java – Gateway
@Component
public class SecurityHeadersFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            HttpHeaders headers = exchange.getResponse().getHeaders();
            headers.set("Strict-Transport-Security",
                "max-age=31536000; includeSubDomains; preload");
            headers.set("Content-Security-Policy", "default-src 'self'");
            headers.set("X-Content-Type-Options", "nosniff");
            headers.set("X-XSS-Protection", "1; mode=block");
            headers.set("Access-Control-Allow-Origin", allowedOrigin);
            // Supprimer les headers d'info serveur
            headers.remove("Server");
            headers.remove("X-Powered-By");
        }));
    }

    @Override public int getOrder() { return -1; }
}
```

## 7. Synthèse — Matrice Endpoint × Mécanisme

Tableau récapitulatif de l'application de chaque mécanisme de sécurité sur chaque endpoint BankID et sur le stockage KYC de la solution.

| Endpoint / Composant                  | TLS 1.3 | QWAC   | QSEAL (vérif) | JWE/JWS      | APub audit           |
|---------------------------------------|---------|--------|---------------|--------------|----------------------|
| GET /openid-configuration             | MUST    | —      | —             | Non          | Non                  |
| GET /.well-known/jwks                 | MUST    | —      | —             | Non          | Non                  |
| GET /auth (redirect)                  | MUST    | SHOULD | —             | Non          | Non                  |
| POST /token                           | MUST    | —      | —             | id_token MAY | MUST (id_token brut) |
| GET /userinfo                         | MUST    | —      | Vérif sig     | MAY          | MUST (payload brut)  |
| GET /profile                          | MUST    | —      | Vérif sig     | MAY          | MUST (payload brut)  |
| POST /revoke                          | MUST    | —      | —             | Non          | Non                  |
| GET /callback (Gateway)               | MUST    | SHOULD | —             | Non          | Non                  |
| GET /.well-known/jwks.json (ton JWKS) | MUST    | SHOULD | —             | Exposé       | Non                  |
| Stockage opérationnel KYC             | —       | —      | —             | —            | AES-256-GCM OK       |
| Stockage audit KYC                    | —       | —      | —             | —            | APub MUST            |
| Logs d'audit (métadonnées)            | —       | —      | —             | —            | Pas de PII           |

### 7.1 Checklist implémentation

1. Configurer WebClient avec TLS 1.3 uniquement, compression désactivée.
2. Importer les CA BankID (root + 2 intermédiaires actifs) dans le TrustStore.
3. Obtenir et configurer un certificat QWAC (FQDN, pas wildcard) sur la Gateway.
4. Exposer un endpoint JWKS HTTPS (/well-known/jwks.json) avec ta clé publique.
5. Implémenter JwtProcessingService : détection JSON/JWT, déchiffrement JWE, vérification JWS.
6. Gérer la rotation automatique des clés JWK (re-fetch sur kid inconnu).
7. Générer la paire APub/APriv — stocker APriv en cold storage (HSM).
8. Implémenter AuditEncryptionService : chiffrement ECDH-ES+A256GCM avec APub.
9. Appeler AuditEncryptionService sur chaque réponse brute /token, /userinfo, /profile.
10. Séparer stockage opérationnel (AES-256-GCM, 30j) et audit (APub, 15 ans).
11. Configurer les headers de sécurité TLS sur la Gateway (HSTS, CSP, nosniff...).
12. Rotation annuelle des certificats QWAC et des clés JWK (obligation MUST).

#### ✓ Conformité doc security\_sep v1.3

Tous les MUST de la doc security\_sep sont couverts dans ce document.

Les SHOULD (QWAC, QSEAL propre) sont documentés avec leur impact.

Le MAY (JWS/JWE) est traité comme MUST côté réception : le code gère les deux cas.

APub/APriv est le point le plus impactant — il nécessite un HSM en production.